

Operation Analytics and Investigating Metric Spike

Advanced SQL

Description:

Operational Analytics is a crucial process that involves Analyzing a company's end-to-end operations. This analysis helps identify areas for improvement within the company. As a Data Analyst, you'll work closely with various teams, such as operations, support, and marketing, helping them derive valuable insights from the data they collect.

One of the key aspects of Operational Analytics is investigating metric spikes. This involves understanding and explaining sudden changes in key metrics, such as a dip in daily user engagement or a drop in sales. As a Data Analyst, you'll need to answer these questions daily, making it crucial to understand how to investigate these metric spikes.

In this project, you'll take on the role of a Lead Data Analyst at a company like Microsoft. You'll be provided with various datasets and tables, and your task will be to derive insights from this data to answer questions posed by different departments within the company. Your goal is to use your advanced SQL skills to analyze the data and provide valuable insights that can help improve the company's operations and understand sudden changes in key metrics.

Case Study 1: Job Data Analysis

You will be working with a table named `job_data` with the following columns:

- **job_id:** Unique identifier of jobs
- **actor_id:** Unique identifier of actor
- **event:** The type of event (decision/skip/transfer).
- **language:** The Language of the content
- **time_spent:** Time spent to review the job in seconds.
- **org:** The Organization of the actor
- **ds:** The date in the format yyyy/mm/dd (stored as text).
- **Data set:**

ds	job_id	actor_id	event	language	time_spent	org
11/30/2020	21	1001	skip	English	15	A
11/30/2020	22	1006	transfer	Arabic	25	B
11/29/2020	23	1003	decision	Persian	20	C
11/28/2020	23	1005	transfer	Persian	22	D
11/28/2020	25	1002	decision	Hindi	11	B
11/27/2020	11	1007	decision	French	104	D
11/26/2020	23	1004	skip	Persian	56	A
11/25/2020	20	1003	transfer	Italian	45	C

Tasks:

A. Jobs Reviewed Over Time:

- Objective: Calculate the number of jobs reviewed per hour for each day in November 2020.
- Your Task: Write an SQL query to calculate the number of jobs reviewed per hour for each day in November 2020.

Code:

```
SELECT ds,sum(time_spent) AS Total_time_spent,  
  
COUNT(job_id) as Number_of_total_jobs,  
  
ROUND((SUM(time_spent) / COUNT(job_id)), 1) AS  
Time_taken_for_each_job_in_secs,  
  
CASE  
  
    WHEN SUM(time_spent) > 0 THEN (SUM(time_spent)/3600 )  
  
    ELSE  
  
        0  
  
    END AS Reviews_in_an_hour_on_each_day_of_November  
  
FROM job_data  
  
GROUP BY ds  
  
ORDER BY Total_time_spent;
```

Result:

ds	Total_time_spent	Number_of_total_jobs	Time_taken_for_each_job_in_secs	Reviews_in_an_hour_on_each_day_of_November
11/29/2020	20	1	20.0	0.0056
11/28/2020	33	2	16.5	0.0092
11/30/2020	40	2	20.0	0.0111
11/25/2020	45	1	45.0	0.0125
11/26/2020	56	1	56.0	0.0156
11/27/2020	104	1	104.0	0.0289

B. Throughput Analysis:

- Objective: Calculate the 7-day rolling average of throughput (number of events per second).
- Your Task: Write an SQL query to calculate the 7-day rolling average of throughput. Additionally, explain whether you prefer using the daily metric or the 7-day rolling average for throughput, and why.

Code:

```
select round(count(event)/sum(time_spent), 2) as Throughput_weekly_basis  
  
from job_data;  
  
select ds as Date, round(count(event)/sum(time_spent), 2) as Throughput_daily_basis
```

from job_data

group by ds

order by ds;

Result:

Date	Throughput_daily_basis
11/25/2020	0.02
11/26/2020	0.02
11/27/2020	0.01
11/28/2020	0.06
11/29/2020	0.05
11/30/2020	0.05

C. Language Share Analysis:

- Objective: Calculate the percentage share of each language in the last 30 days.
- Your Task: Write an SQL query to calculate the percentage share of each language over the last 30 days.

Code:

```
select language, round(100*count(*)/total,2) as Percentage, sub.total from job_data
cross join(
select count(*) as total from job_data) as sub
group by language, sub.total;
```

Result:

language	Percentage	total
English	12.50	8
Arabic	12.50	8
Persian	37.50	8
Hindi	12.50	8
French	12.50	8
Italian	12.50	8

D. Duplicate Rows Detection:

- Objective: Identify duplicate rows in the data.
- Your Task: Write an SQL query to display duplicate rows from the job_data table

Code:

```
with cte as(
select actor_id,ds,job_id,event,language,time_spent,org, row_number()
over(partition by actor_id,ds,job_id,event,language,time_spent,org
order by actor_id) as RN
from job_data)
select * from cte where RN >1;
```

-- Approach two:-

```
select actor_id, count(*) as Duplicates from job_data  
group by actor_id having count(*)>1;
```

Result:

	actor_id	Duplicates
▶	1003	2

Case Study 2: Investigating Metric Spike

You will be working with three tables:

- **users:** Contains one row per user, with descriptive information about that user's account.
- **events:** Contains one row per event, where an event is an action that a user has taken (e.g., login, messaging, search).
- **Email events:** Contains events specific to the sending of emails.

Tasks:

A. Weekly User Engagement:

- Objective: Measure the activeness of users on a weekly basis.
- Your Task: Write an SQL query to calculate the weekly user engagement.

Code:-

```
SELECT * FROM email_events;
```

```
SELECT
```

```
WEEK(STR_TO_DATE(occurred_at, '%d-%m-%Y %H:%i')) AS Weeks,
```

```
COUNT(DISTINCT user_id) AS Engaged_user
```

```
FROM email_events
```

```
WHERE action = "sent_weekly_digest"
```

```
GROUP BY Weeks
```

```
ORDER BY Weeks;
```

-- Note:

```
-- EXTRACT(WEEK FROM STR_TO_DATE(occurred_at, '%d-%m-%Y %H:%i')) AS  
Weeks.alter
```

Above code can also be used to get the same result. EXTRACT(WEEK FROM ...) is a general-purpose function that can extract various date parts (e.g., year, month, day) from a date. By specifying WEEK, you instruct it to extract the week number from the date.

But in our question set, they only asked for the weeks, that's why we use WEEK(), which is specifically designed to extract only the week number from a date.

Result:-

Result Grid			Filter R
	Weeks	Count_of_User	
▶	17	908	
	18	2602	
	19	2665	
	20	2733	
	21	2822	
	22	2911	
	23	3003	
	24	3105	
	25	3207	
	26	3302	
	27	3399	
	28	3499	
	29	3592	
	30	3706	
	31	3702	

B. User Growth Analysis:

- Objective: Analyze the growth of users over time for a product.
- Your Task: Write an SQL query to calculate the user growth for the product.

Code:

```
SELECT
    YEAR(STR_TO_DATE(created_at, '%d-%m-%Y %H:%i')) AS Year,
    MONTH(STR_TO_DATE(created_at, '%d-%m-%Y %H:%i')) AS Month,
    WEEK(STR_TO_DATE(created_at, '%d-%m-%Y %H:%i')) AS Week,
    COUNT(DISTINCT user_id) AS New_Users,
    SUM(COUNT(DISTINCT user_id)) OVER
    (ORDER BY Year(STR_TO_DATE(created_at, '%d-%m-%Y %H:%i'))
     ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS
    Sum_of_newly_joined_users
FROM
    users
GROUP BY
    Year,Month,Week
ORDER BY
```

Year;

Result:

	Year	Month	Week	New_Users	Sum_of_newly_joined_users
►	2013	1	0	23	23
	2013	1	1	30	53
	2013	1	2	48	101
	2013	1	3	36	114
	2013	1	4	23	107
	2013	2	4	7	66
	2013	2	5	48	78
	2013	2	6	38	93
	2013	2	7	42	128
	2013	2	8	25	105
	2013	3	8	9	76
	2013	3	9	43	77
	2013	3	10	32	84
	2013	3	11	31	106
	2013	3	12	22	86

Note:

In some places, the number of newly joined users has decreased, From the above table, we can see that the number of freshly joined users has decreased from 101 to 66 between weeks 2 and 3, from 107 to 76 between weeks 3 and 4, from 128 to 105 between weeks 3 and 4, from 106 to 84 between weeks 3 and 4, and from 106 to 84 between weeks 3 and 4. In all other cases, the number of newly joined users has increased from the previous week. Now companies can find out the specific area where they lose their customers, and work accordingly.

C. Weekly Retention Analysis:

- Objective: Analyze the retention of users on a weekly basis after signing up for a product.
- Your Task: Write an SQL query to calculate the weekly retention of users based on their sign-up cohort.

Code:

SELECT

WEEK(STR_TO_DATE(occurred_at, '%d-%m-%Y %H:%i')) AS Week,

COUNT(DISTINCT user_id) AS Total_users

FROM events

WHERE event_type = 'signup_flow' AND event_name = 'complete_signup'

GROUP BY Week

ORDER BY Week;

Result:

	Week	Total_users
▶	17	72
	18	163
	19	185
	20	176
	21	183
	22	196
	23	196
	24	229
	25	207
	26	201
	27	222
	28	215
	29	221
	30	238
	31	193
	32	52

Note:

From Week 17 to Week 32, we observe fluctuations in user retention, with a steady increase in total users, peaking around Week 30, followed by a noticeable decline through Week 32. This pattern indicates that user engagement may drop off after Week 30. To improve retention, we can analyze factors contributing to these drop-off points and implement targeted strategies to sustain engagement. By identifying specific points where users disengage, we can enhance the user experience and develop effective retention tactics to maintain a steady user base.

D. Weekly Engagement Per Device:

- Objective: Measure the activeness of users on a weekly basis per device.
- Your Task: Write an SQL query to calculate the weekly engagement per device.

Code:

```
SELECT
device,
WEEK(STR_TO_DATE(occurred_at, '%d-%m-%Y %H:%i')) AS Week,
COUNT(DISTINCT user_id) AS Number_of_users
FROM events
WHERE event_type = 'engagement'
GROUP BY device, Week
ORDER BY Week;
```

Result:

	device	Week	Number_of_users
▶	acer aspire desktop	17	2
	acer aspire notebook	17	2
	amazon fire phone	17	1
	asus chromebook	17	3
	dell inspiron desktop	17	1
	dell inspiron notebook	17	4
	hp pavilion desktop	17	2
	htc one	17	2
	ipad air	17	1
	ipad mini	17	3
	iphone 4s	17	3
	iphone 5	17	11
	iphone 5s	17	5
	lenovo thinkpad	17	8
	mac mini	17	1
	macbook air	17	4
	macbook pro	17	13
	nexus 5	17	4
	nexus 7	17	4
	nokia lumia 635	17	2

Note:

"We are tracking weekly user activity by device to assess engagement levels. The data shows that user engagement varies across different devices, with some devices consistently having higher engagement than others. To improve engagement, we can analyze factors affecting these variations, focusing on device compatibility and optimizing the user experience. Additionally, monitoring trends for devices with lower engagement will help us identify specific challenges, allowing us to develop targeted strategies to enhance engagement on these devices."

E. Email Engagement Analysis:

- Objective: Analyze how users are engaging with the email service.
- Your Task: Write an SQL query to calculate the email engagement metrics.

Code:

SELECT

DATE_FORMAT(STR_TO_DATE(occurred_at, '%d-%m-%Y %H:%i'), '%Y-%m') AS Month,

COUNT(CASE WHEN action = 'sent_weekly_digest' THEN 1 END) AS Emails_Sent,

COUNT(DISTINCT CASE WHEN action = 'email_open' THEN user_id END) AS
Unique_Openers,

COUNT(CASE WHEN action = 'email_clickthrough' THEN 1 END) AS Total_Clicks,

COUNT(DISTINCT CASE WHEN action = 'email_clickthrough' THEN user_id END) AS
Unique_Clickers,

CASE

WHEN COUNT(DISTINCT CASE WHEN action = 'email_open' THEN user_id END) > 0

```

        THEN ROUND(COUNT(DISTINCT CASE WHEN action = 'email_clickthrough' THEN
user_id END) /

        COUNT(DISTINCT CASE WHEN action = 'email_open' THEN user_id END) * 100,
2)

        ELSE 0

        END AS Click_Through_Rate

FROM

        email_events

GROUP BY

        Month

ORDER BY

        Month;

```

Result:

Month	Emails_Sent	Unique_Openers	Total_Clicks	Unique_Clickers	Click_Through_Rate
2014-05	11730	2681	2023	1703	63.52
2014-06	13155	3037	2274	1915	63.06
2014-07	15902	3457	2721	2267	65.58
2014-08	16480	3879	1992	1804	46.51

Note:

The engagement data for the weekly digest emails between May and August 2014 shows positive trends with some fluctuations:

1. **Emails Sent:** Monthly email volume increased from 11,730 in May to 16,480 in August, suggesting a growing user base or enhanced outreach.
2. **Unique Openers:** Unique openers also rose, from 2,681 in May to 3,879 in August, reflecting increased engagement and interest among users.
3. **Clicks and Clickers:** Both total clicks and unique clickers peaked in July but dropped in August, with clicks decreasing from 2,721 to 1,992, and unique clickers from 2,267 to 1,804.
4. **Click-Through Rate (CTR):** The CTR remained strong, above 60% in most months, peaking at 65.58% in July. However, August showed a notable decrease to 46.51%, suggesting possible user fatigue or content relevance issues.

Please note that for each task, you should also provide insights and interpretations of the results obtained from your queries.

How to do this Project?

1. **Create the Database and Tables:** Start by creating a database for the project and then create the necessary tables using the provided table structures and links. You need to import the csv file into MySQL Workbench.

2. **Perform Analysis:** Utilize SQL to perform the analysis and answer the questions mentioned in the case studies. Ensure that you understand the table structures and the meaning of various columns such as event types and what to consider for reviewing.
3. **Submit a Report:** Prepare a report (PDF/PPT) to present your findings to the leadership team. The report should include the following details:

Project Description

Provide a brief overview of the project, explaining its purpose and how you plan to handle the analysis.

Approach

Describe your approach towards the project and explain how you executed the analysis.

Tech-Stack Used

Mention the software and versions used for the project, such as MySQL Workbench, and explain their purpose in the analysis.

Insights

Summarize the insights and knowledge you gained during the project. Highlight key observations and inferences you made from the data, keeping them concise and to the point.

Result

Discuss what you have achieved through the project and explain how it has contributed to your understanding and decision-making.

Drive Link

Save your report as a PDF file and upload it to your Google Drive. Include the shareable link (with public visibility) in your report. Check the detailed steps below for submission.

Note: Make sure you mention the SQL Queries along with the outputs in the report. You can take snapshots of the SQL Queries and the outputs. It is advised to do the project in MySQL Workbench.

How to Submit your Project:

Step 1 Do your project in PPT/PDF. It's your choice. For PDF, write your project in Ms Word, then save the file in PDF format. You will find the option in the 'Save As' Box.

Step 2 Now, upload the file i.e., PPT/PDF on your google drive. It would be wise to create a Single Folder and Name it "Trainity Assignments". Inside that folder, create another Folder name "Project Name". Upload your assignment there.

Step 3 Right Click on the File, and select 'Get Link' under 'Share'. A Dialog Box will Pop up. Inside the Box, Under General Access, Change 'Restricted' to 'Anyone with the Link'. No need to change anything else.

Step 4 Click on 'Copy Link'.

Step 5 Now paste the link in the Drive Link box when submitting the project.

Judgement Criteria:

SQL Understanding

The SQL queries used should provide correct outputs and be easily understandable. Ensure that the queries are accurate, efficient, and well-structured.

Case study completion

All the questions presented in the case studies must be answered completely with accurate responses. Address each question thoroughly, demonstrating your understanding of the analysis required.

Insights

Use your own imagination and creativity to answer the case study questions, providing insightful and meaningful observations and recommendations. Think beyond the obvious and showcase your ability to derive valuable insights from the data.

Plagiarism

The project should be your own original work and should not be copied from the internet or any other source. Ensure that the analysis, interpretations, and visualizations are your own, demonstrating your skills as a data analyst.

By following these guidelines and demonstrating your SQL understanding, completing the case studies, providing insightful observations, and avoiding plagiarism, you will showcase your expertise and deliver a comprehensive and original SQL project.